

ALG-Tema3-EJs_DyV.pdf



Anónimo



Algorítmica

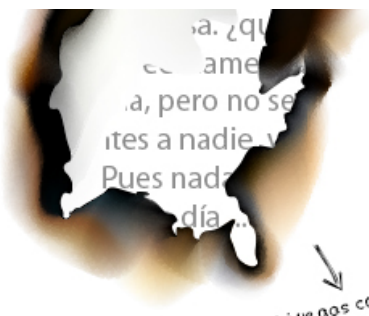


2º Grado en Ingeniería Informática



Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación
Universidad de Granada

NO
QUEMES
TUS
APUNTES



si juegas con fuego
te fuegas

**GANAR
0,25 €**

por subir tus apuntes
en PDF a Wuolah

QUIERES 15€ ?



TRAER A TU CRUSH
DE APUNTES ♡



WUOLAH

si consigues
que suba
apuntes, te
llevaras 15€
+ 5 Wuolah
Coins para
los próximos
sorteos

- ① Sea A un array bidimensional de tamaño $n \times n$ parcialmente ordenado (se supone n potencia de 2). (pág 1)
El criterio de ordenación es el siguiente: Los elementos en cada fila y columna se encuentran en orden ascendente, esto es,
- $A[i, j] \leq A[i, j+1]$ con $i=1, \dots, n$ y $j=1, \dots, n$.
 - $A[i, j] \leq A[i+1, j]$ con $i=1, \dots, n$ y $j=1, \dots, n$.

Se pretende determinar si un elemento está en la matriz. Responder a las siguientes cuestiones:

- Construir un algoritmo básico (ad-hoc).
- Resolver de manera eficiente con Divide y Venceras.
- Comprobar si los órdenes mejoran.

Matriz de ejemplo,

20	25	26	29
22	27	33	40
29	39	41	45
31	43	50	55

→ comienzo del
Alg. Básico.

- a) Aprovechando la información que tenemos de la matriz podemos ver (sea x el elemento a buscar):

- Si $x = A[i, j] \rightarrow$ encontrado.
- Si $x < A[i, j] \rightarrow$ puedo mirar en $A[i, j-1]$
- Si $x > A[i, j] \rightarrow$ puedo pasar a $A[i+1, j]$

Empezando en la posición $i=0$ y $j=n-1$ puedo ir descartando filas y columnas hasta encontrar el elemento o llegar a la posición $i=n$ o $j=-1$.

El algoritmo detallado sería,

si juegas con
Fuego te Fuegos



WUOLAH

(pág 2)

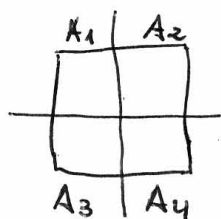
```

int busqueda (int n, int x, int **A) {
    int f, c;
    f = 0;
    c = n-1;
    while ((f <= n-1) && (c >= 0))
        if (x < A[f][c]) c = c-1;
        else
            if (x > A[f][c]) f = f+1;
            else return 1;
    return 0;
}

```

En el peor caso (cuando no se encuentra el elemento), la eficiencia es $2 \cdot n \rightarrow O(n)$.

b) Para aplicar la técnica DV podemos encontrar el elemento medio de la matriz (como se hacía en búsqueda binaria) y ver en qué parte o partes de la matriz tengo que seguir buscando. De esta forma,



si $x = A[n/2, n/2] \rightarrow$ encontrado.

si $x < A[n/2, n/2] \rightarrow$ hay que buscar en A_1, A_2 y A_3 .

si $x > A[n/2, n/2] \rightarrow$ hay que buscar en A_2, A_3 y A_4 .

El caso base sería cuando sólo quede un elemento, momento en el cual podré devolver verdadero o falso hacia arriba. Si alguno de los subcasos da verdadero, se devuelve verdadero. El algoritmo de talleo sería, verdadero, si no, falso.

QUIERES
CONSEGUIR
15€??

TRÁENOS A TU
CRUSH DE APUNTES
ANTES DE QUE
LOS QUEME



si consigues que suba apuntes, te llevas 15€ + 5
Wuolah Coins para los sorteos

WUOLAH


```

int busqueda (int fin, int cini, int ffin, int cfin, int x, int **A) {
    int fmed, cmed;
    if (ffin - fin == 0)
        if (x == A[fin][cini]) return 1;
        else return 0;

    fmed = fin + (ffin - fin + 1) / 2 - 1;
    cmed = cini + (cfin - cini + 1) / 2 - 1;
    if (x == A[fmed][cmed]) return 1;
    if (busqueda (fmed + 1, cini, ffin, cmed, x, A)) return 1;
    if (busqueda (fin, cmed + 1, fmed, cfin, x, A)) return 1;
    if (x < A[fmed][cmed]) {
        if (busqueda (fin, cini, fmed, cmed, x, A)) return 1;
    }
    else
        if (busqueda (fmed + 1, cmed + 1, ffin, cfin, x, A)) return 1;
    return 0;
}

```

Diagrama de flujo de la función `busqueda`:

- Se calcula `fmed` y `cmed` como el punto medio de la submatriz.
- Se verifica si `x == A[fmed][cmed]`. Si es cierto, se devuelve 1.
- Se llama recursivamente a `busqueda` en la submatriz inferior derecha (`fmed+1, cini, ffin, cmed`).
- Se llama recursivamente a `busqueda` en la submatriz superior izquierda (`fin, cmed+1, fmed, cfin`).
- Si `x < A[fmed][cmed]`, se llama recursivamente a `busqueda` en la submatriz inferior izquierda (`fin, cini, fmed, cmed`).
- Si `x > A[fmed][cmed]`, se llama recursivamente a `busqueda` en la submatriz superior derecha (`fmed+1, cmed+1, ffin, cfin`).
- Si no se encuentra, se devuelve 0.

En el peor de los casos la función de tiempo está definida por,

$$T(n) = 3T(n/2) + 1 \Rightarrow l = 3 > b^k = 2^0 = 1,$$

$$O(n^{\lg_b l}) = O(n^{\lg 3})$$



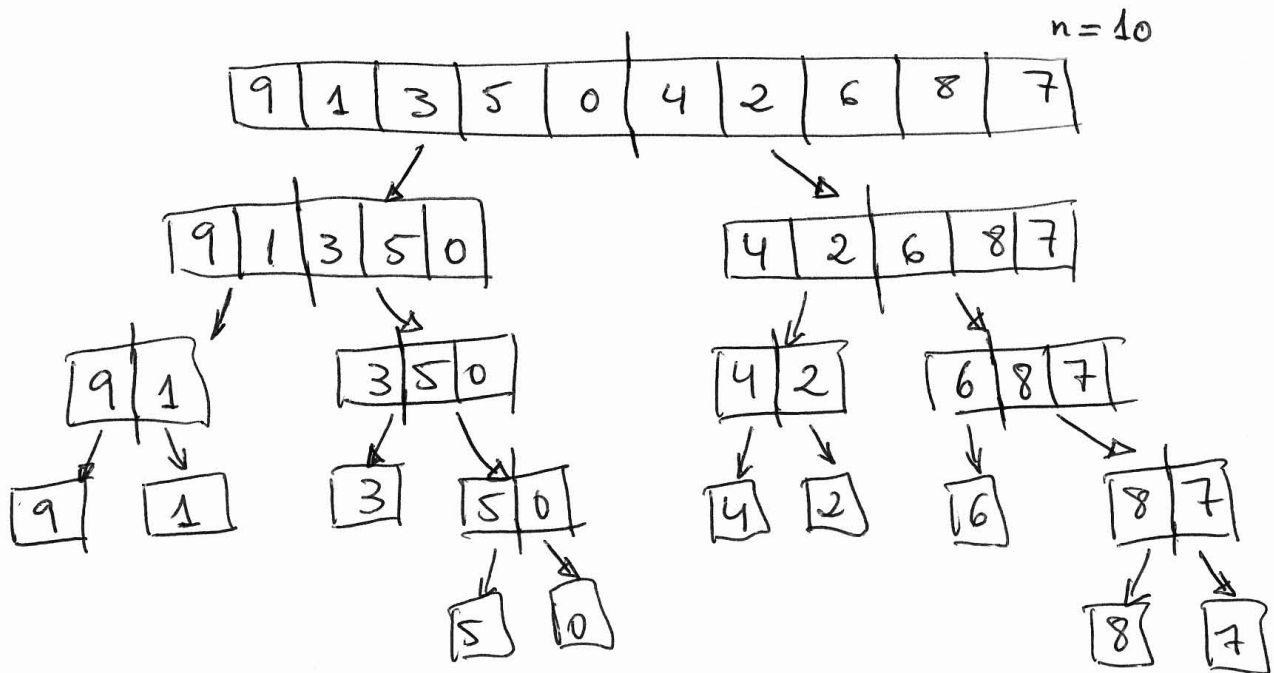
El orden no se mejora al aplicar DyV.

② Ordenar los siguientes elementos utilizando Mergesort y Quicksort:

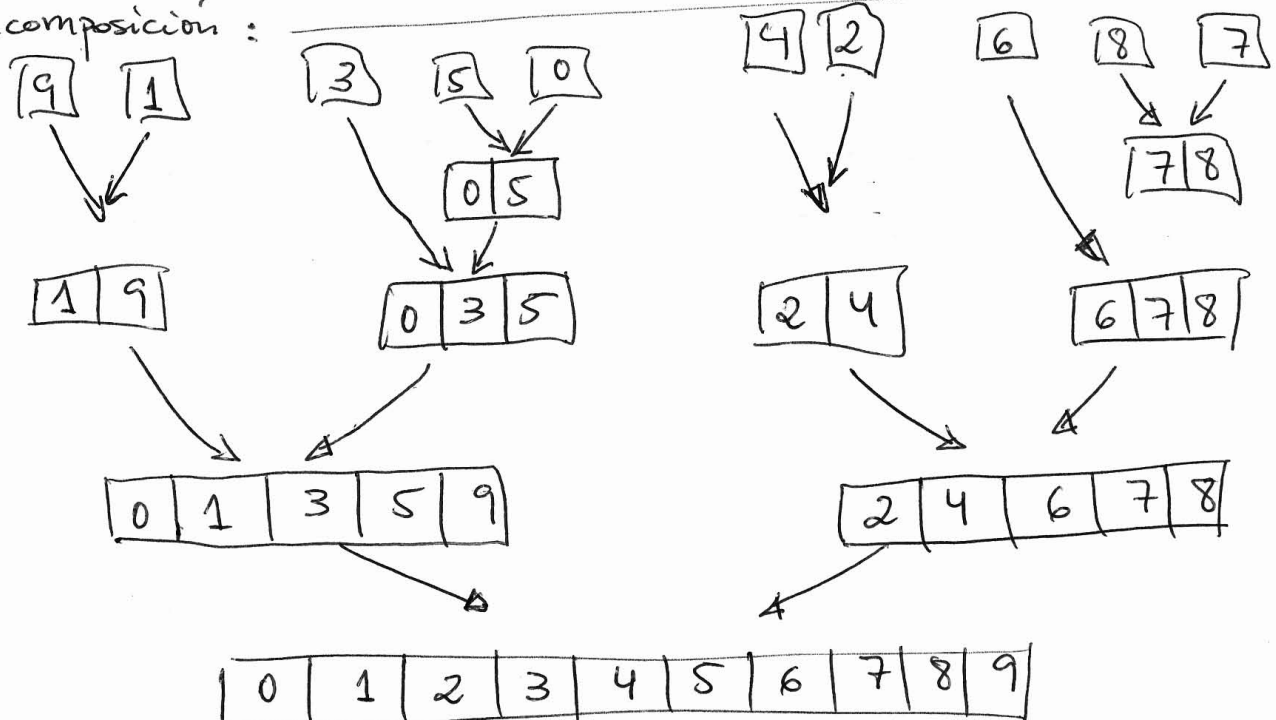
pág 4

9 1 3 5 0 4 2 6 8 7

Merge:



Recomposición:



QUIERES 15€ ?

TRAER A TU CRUSH DE APUNTES ♡



WUOLAH

si consigues que suba apuntes, te llevas 15€ + 5 Wuolah Coins para los próximos sorteos

Quick sort :

pivote

pág 5

posiciones del vector

9	1	3	5	0	4	2	6	8	7
0	1	2	3	4	5	6	7	8	9

(elijo como pivote el elemento medio del vector)
(utilizo pivoteo lineal con dos índices e incluyendo el pivote como elemento de la parte de la derecha).

balanceo

9	1	3	5	0	4	2	6	8	7
0	1	2	3	4	5	6	7	8	9

izqda. vacía

vuelvo a tener el mismo vector y se repite la situación.

Necesito cambiar algo (la elección del pivote junto con el balanceo utilizado no asegura que se reduzca el vector en todos los casos). Soluciones:

- Excluir el pivote del balanceo, sacándolo al principio y luego llevándolo a su posición.
- Elegir un pivote que asegure que haya al menos un elemento en la parte de la izquierda (el mayor de los dos primeros).

Considerando la segunda opción tendríamos:

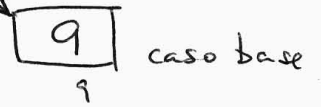
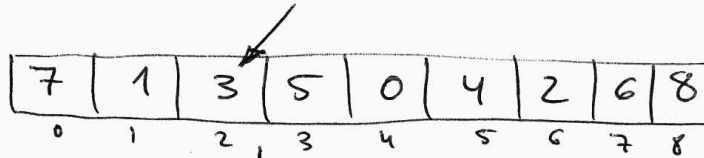
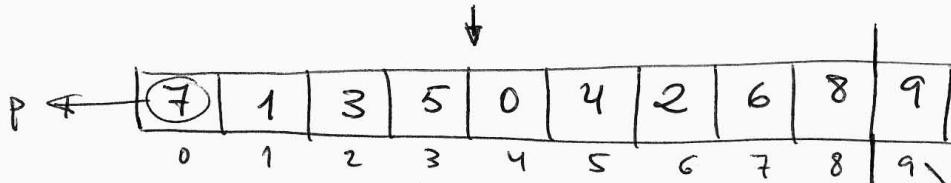
p ← 9	1	3	5	0	4	2	6	8	7
0	1	2	3	4	5	6	7	8	9

- El puntero derecho. para en el 7.
- El puntero izqda. para en el 9.
- Intercambio el 7 con el 9.
- drcha. para en 8.
- Izqda. se cruza con drcha.

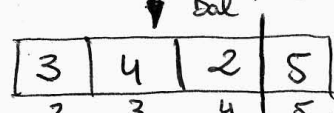
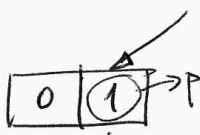
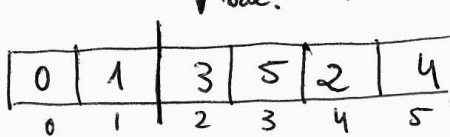
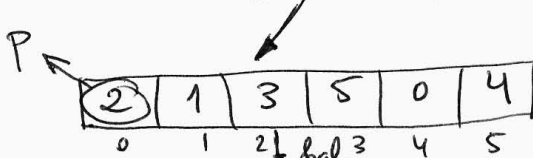
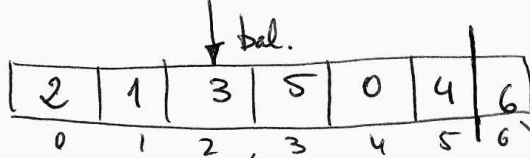
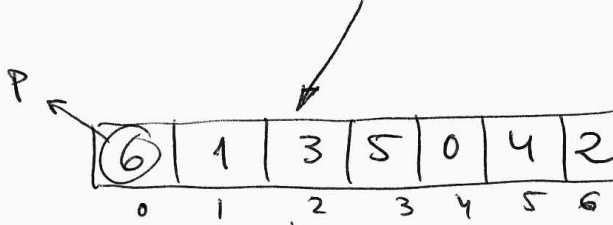
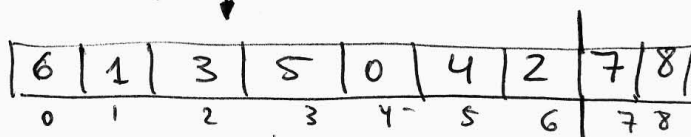
si juegas con fuego te quemas



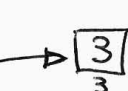
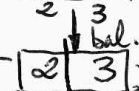
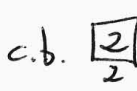
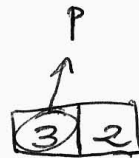
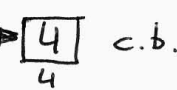
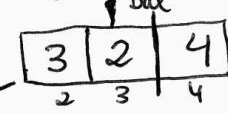
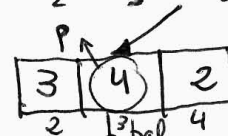
WUOLAH



- drcha. para en 6.
- Izqda. en 7.
- Intercambio 7 y 6.
- Drcha. para en 2
- Izqda. se cruza con drcha.



Una forma inteligente para hacer el ejercicio más rápido hubiese sido indicar que el pivote se escoge de forma aleatoria e ir eligiendo un elemento que dividiera bien el vector.



- ③ Encontrar la mediana de un vector de tamaño n . Se supone que los elementos no están repetidos. En el caso de que n sea par, se considerará como mediana el elemento anterior a la mitad del vector. (pág 7)

El algoritmo básico podría consistir en ordenar el vector y quedarnos con el elemento medio. Tendría orden $O(n \cdot \lg n)$.

Para aplicar D y V escogeríamos un pivote para balancear los elementos a derecha e izquierda como en Quicksort. Seguiríamos buscando únicamente por la parte en la cual quedara el elemento medio. Una vez llegado al caso base, devolveríamos dicho elemento como valor de la mediana (el caso base se produce cuando sólo queda un elemento por explorar).

El algoritmo detallado sería,

```
int n-esimo (int pos, int ini, int fin, int *V) {
    int l;
    if (fin - ini == 0) return V[ini];
    l = pivot (ini, fin, V);
    if (l == pos) return V[l];
    if (pos > l)
        return n-esimo (pos, l + 1, fin, V);
    else
        return n-esimo (pos, ini, l - 1, V);
}
```

(porque el pivote no se va a incluir ni en la parte drcha. ni en la parte izqda.)

Se llamaría a la función de la siguiente manera:

$mediana = n-esimo(n / 2, 0, n - 1, V);$

Véase que con esta función se puede obtener el elemen

to que esté situado en cualquier posición.

La función pivot habría que modificarla para que devolviera la posición en la que queda el pivote. Quedaría de la siguiente manera,

```
int pivot (int ini, int fin, int *V) {
    int p, k, l, temp;
    p = V[ini];
    k = ini;
    l = fin + 1;
    do {
        k++;
        while (V[k] <= p && k < fin);
        do {
            l--;
        } while (V[l] > p);
        while (k < l) {
            temp = V[k];
            V[k] = V[l];
            V[l] = temp;
            do {
                k++;
            } while (V[k] <= p);
            do {
                l--;
            } while (V[l] > p);
        }
        temp = V[ini];
        V[ini] = V[l];
        V[l] = temp;
        return l;
    }
}
```

El orden se calcularía considerando $T(n) = T(n/2) + n$
 $l=1, k=1, b=2 \Rightarrow l < b^k \rightarrow 1 < 2^1 \Rightarrow$ el orden es $O(n^1)$

QUIERES 15€ ?



TRAER A TU CRUSH
DE APUNTES ♡



WUOLAH

si consigues
que suba
apuntes, te
llevarás 15€
+ 5 Wuolah
Coins para
los próximos
sorteos

④ (Ej. 1 del examen de Febrero de 2007) (1'5 pt).

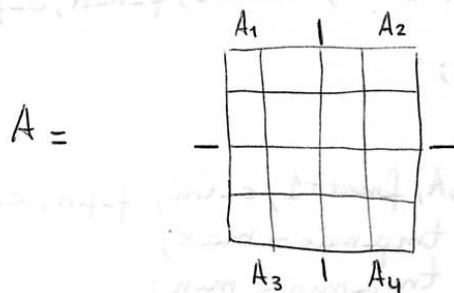
pag. 9

Dada una matriz cuadrada de enteros de dimensión $n \times n$, se desea encontrar el máximo y mínimo de la misma.

- Diseñar un algoritmo basado en la técnica "Divide y Vencerás" para resolver este problema.
- Calcular la eficiencia del algoritmo en el peor caso.

Suponer que n es potencia de algún número entero conveniente.

- Supongamos n potencia de 2. La matriz se puede dividir en 4 matrices de tamaño $n/2$. Ej:



El caso base se daría cuando la matriz tenga tamaño 1. En ese caso se devolvería el único valor disponible como máximo y como mínimo.

Para recomponer a partir de 4 subproblemas nos quedamos con el máximo y el mínimo de los 4 valores propuestos para cada subproblema. Desgraciadamente no se puede decrementar el número de llamadas, ya que los valores pueden estar en cualquiera de las partes.

El algoritmo detallado sería

si juegas con
Fuego te Fuegos




```

void MaxMinDV (int &max, int &min, int **A,
               int f-ini, int c-ini, int f-fin, int c-fin) {
    // se numera de 0 a n-1
    int f-med, c-med, tmp-max, tmp-min;
    if (f-fin - f-ini == 0) { // caso base
        max = min = A[f-ini][c-ini];
        return;
    }
    f-med = (f-ini + f-fin) / 2; // división entera
    c-med = (c-ini + c-fin) / 2;
    MaxMinDV (tmp-max, tmp-min, A, f-ini, c-ini, f-med, c-med); // A1
    MaxMinDV (max, min, A, f-ini, c-med+1, f-med, c-fin); // A2
    if (max > tmp-max)
        tmp-max = max;
    if (min < tmp-min)
        tmp-min = min;
    MaxMinDV (max, min, A, f-med+1, c-ini, f-fin, c-med); // A3
    if (max > tmp-max) tmp-max = max;
    if (min < tmp-min) tmp-min = min;
    MaxMinDV (max, min, A, f-med+1, c-med+1, f-fin, c-fin); // A4
    if (tmp-max > max) max = tmp-max;
    if (tmp-min < min) min = tmp-min;
    return;
}

```

b) La ecuación de tiempo recurrente se puede establecer de la siguiente manera:

$$T(n) = 4 T(n/2) + 1,$$

$$l=4, k=0, b=2 \Rightarrow l > b^k \rightarrow 4 > (2^0=1) \Rightarrow$$

$$\Rightarrow \text{el orden es } O(n^{\lg 4}) = O(n^2)$$

Este orden no mejora al del algoritmo básico (recorrer todos los elementos por filas y columnas) que es $O(n^2)$.

Para realizar la llamada se puede utilizar una función que haga de interfaz:

```
void MaxMin (int &max, int &min, int **A, int n) {  
    MaxMinDV (max, min, A, 0, 0, n-1, n-1);  
    return;  
}
```

⑤

(Ej. 4 del examen de Septiembre de 2007) (2 pt.).

Dado un vector $T[1..n]$ se dice que un elemento del vector x es mayoritario si aparece en el vector un número de veces estrictamente mayor de $n/2$. Diseñar un algoritmo para decidir si dado un vector tiene un elemento mayoritario y, en su caso, indicar cuál es ese elemento.

Para resolver este problema se puede proceder de distintas formas:

- 1) Un algoritmo básico consistiría en recorrer el array y para cada elemento contar el número de ocurrencias de dicho elemento. Sólo habría que llegar hasta la posición $(n+1)/2$, quedándonos con el elemento que más ocurrencias tiene y devolviéndolo si el número de ocurrencias es mayor que $n/2$. Este algoritmo es $O(n^2)$.
- 2) Reformular el problema en dos partes: primero identificar el único elemento que podría ser mayoritario, segundo contar sus ocurrencias para ver si lo es. Se basa en la idea de que el elemento mayoritario debería estar en la posición $(n+1)/2$.

si el vector estuviera ordenado. Se puede resolver de dos maneras:

pág 12

a) Ordenar con Quicksort y contar el número de ocurrencias del elemento que hay en la posición $(n+1)/2$. Sería $n \cdot \lg n + n \Rightarrow O(n \cdot \lg n)$. Este algoritmo mejora la eficiencia del algoritmo básico.

b) Sin embargo, hay una forma mejor de proceder. Recordemos que se propuso un algoritmo DyV para calcular la mediana de un vector (ejercicio 3, pág. 7). En realidad el algoritmo que se propuso era capaz de calcular el elemento n -ésimo de cualquier vector. Bastaría con usar dicha función para calcular el elemento $(n+1)/2$ -ésimo y contar el número de ocurrencias de dicho elemento. Sería $n + n \Rightarrow O(n)$.

```
int mayoritario (int & x, int * T, int n) {  
    int cont = 0, i;  
    x = n-esimo ((n+1)/2, 0, n-1, T);  
    for (i = 0; i < n; i++)  
        if (T[i] == x)  
            cont++;  
    if (cont > n/2)  
        return 1;  
    return 0;  
}
```

Para responder a este ejercicio habría que indicar o describir cómo se diseña el algoritmo n -ésimo.